

Notes on EventStorming

Introduction

Domain-Driven Design

Complexity is our enemy

Strategic and Tactical Design

Two open questions

EventStorming

Big Picture EventStorming

Preparations

Standard steps

Chaotic exploration

Enforce the timeline

Pivotal events & Swimlanes

Hotspots

People & Systems

Explicit walkthrough

Custom steps

Outcome

Process Modeling EventStorming

Assumptions

The Team

Notation

Grammar

Game Rules

Outcome

Software Design EventStorming

Game Rules

Defining Aggregates

Bounded Contexts

Outcome

Remote EventStorming

EventStorming in COVID-19 times

References

Introduction

Domain-Driven Design

Domain-Driven Design is a structured software design process — one of many — that places the business domain at the center of the design process, ensuring the software model reflects the real-world problem space faithfully.

The original ideas come from the famous “Blue Book” by Eric Evans, published in the early 2000s. Over the last decade, DDD has heavily influenced microservices, which is the most widely used architecture for large-scale systems.

And that is the key word: *large*. DDD is typically applied in organizations dealing with huge, complex business domains. You don't usually see it in smaller companies or early-stage startups — maybe a lighter version, but not the full thing.

The reason comes down to one word: **complexity**.

Complexity is our enemy

Complexity is anything related to the structure of a software system that makes it hard to understand and modify the system. — John Ousterhout,
A philosophy of software design.

It is the **number one enemy of software**, and our goal as software engineers is to keep it **as low as possible**.

In large software systems, the primary source of complexity is not the technology. It is the domain itself: the business rules, the workflows, the edge cases, the terminology, the organizational dynamics.

There are two kinds of complexity:

- **Essential complexity** is inherent to the business problem. We cannot eliminate it — we can only manage it.
- **Accidental complexity** is introduced by poor design choices, misunderstood requirements, and broken communication. This must be minimized.

The irony is that most software projects produce enormous accidental complexity precisely in the domain layer. The result is a set of familiar failure modes:

- **Communication gap.** Developers and domain experts speak fundamentally different languages. Requirements are mistranslated into code, and misalignments accumulate silently over time.
- **Anemic domain model.** Objects exist, but they are pure data containers — getters, setters, and nothing else. Business logic leaks into service classes and procedural scripts scattered across the codebase. The business rules have no home. They cannot be found, reasoned about, or tested in isolation. They are everywhere and nowhere.
- **Big ball of mud.** Without explicit structure, every part of the system becomes entangled with every other. Changes in one place ripple unpredictably to another, and the cost of any modification grows with every release.
- **Implicit knowledge.** The most dangerous knowledge in any organization is the knowledge that exists only in someone's head. Business rules that are not written down, edge cases handled by habit, policies that everyone knows but no one has articulated — all of this is invisible to the system. When that person leaves, the knowledge disappears. The code begins to diverge from the business.

Strategic and Tactical Design

DDD provides a set of patterns, best practices, and principles to address domain complexity. It is not just a philosophy, it is also a toolkit.

More specifically, there are two different layers: **Strategic Design** and **Tactical Design**.

Strategic Design belongs to the *problem space*. Here, we focus on high-level analysis of the domain, staying away from technical details. No code, no architecture, no tech stack.

What we do in this phase is identify subdomains within the larger domain and establish clear boundaries between them. Why? Because we cannot model large systems with a single, unified domain model. Different parts of the business use the same words to mean different things. A *Customer* in the sales department is not the same concept as a *Customer* in billing or support. So we split the domain into smaller parts and — most importantly — define the boundaries between them. We call these **Bounded Contexts**. Inside the boundary, terms have one meaning. Outside it, the same words may mean something different.

We also develop a shared language — called **Ubiquitous Language** — within each Bounded Context. This is arguably the single most important idea in DDD: a living vocabulary, co-created by developers and domain experts, that evolves as the team's understanding of the domain deepens. Terms must be agreed, precise, and reflected in the code.

Finally, we need to think about how Bounded Contexts relate to each other. This is what a **Context Map** captures — not just a technical diagram, but a political one: it reveals which teams depend on which, who has power over shared interfaces, and where friction is most likely to occur.

With Tactical Design, we move into the *solution space*. Think of it as a set of building blocks for implementing a domain model inside a single Bounded Context. We won't cover every building block in these notes, but the one you should keep in mind is the **Aggregate**: a cluster of related objects treated as a single consistency unit, controlled by an Aggregate Root that enforces all invariants.

Two open questions

Applying DDD in practice raises two uncomfortable questions.

Should you trust the domain experts? After reading the Blue Book, it is tempting to believe that domain experts are willing and able to share the knowledge we need. That is not what happens in reality. In large organizations, domain experts live in *silos*. They are genuinely expert in their own area, but they have little understanding of what happens elsewhere. The knowledge they share is locally consistent, not globally coherent. We are not simply translating business requirements into software — the knowledge about the problem domain is fragmented, contested, and often not ready to be turned into code.

How do we spot Bounded Contexts? The concept is clear in theory. Finding the boundaries in a real organization is another matter. There is a relationship between Bounded Contexts and the fragmentation of knowledge inside an organization. But identifying where one context ends and another begins requires more than theory.

The more complex the domain, the more the real challenge shifts away from writing code. Tools and IDEs help us write software faster. Nothing helps us write the *right* software.

This is where EventStorming starts to help.

EventStorming

EventStorming is a *family* of **collaborative modeling workshop formats** that allows you to explore complex domains effectively and engagingly.

The original proposition introduced EventStorming as a single format, but over time it evolved into three distinct and widely adopted variants:

1. **Big Picture EventStorming**
2. **Process Modeling EventStorming**
3. **Software Design EventStorming**

You can use each format independently or combine them into a progressive flow.

A common approach is to start with Big Picture EventStorming to identify the areas with the highest improvement potential, and then zoom in using Process Modeling EventStorming.

In some cases, a more detailed format such as Software Design EventStorming naturally extends the conversation, adding technical depth without losing the business-centric perspective.

Let's take a closer look at each format.

Big Picture EventStorming

Big Picture EventStorming is a large-scale workshop aiming to discover the intricacies of an entire business line — not a single feature, not a single team's perspective, but the full picture.

The structure is **incremental**. Each step adds a new layer of information: a raw timeline of events, then people and systems, then a validated narrative. The model starts thin and grows richer as more perspectives enter the room.

The main purpose is to **maximize learning**. Participants from different departments — often working in silos, rarely talking to each other — build a model together. Inconsistencies surface automatically. Problems that were invisible become visible. And by the end, everyone in the room has agreed, at least once, on what the system actually does and which problems are most critical to solve.

Preparations

1. Invite **the right people**:
 - *Experts and explorers*, ideally a mix of business, IT and UX people.
 - Ensure the group includes people who can both answer and ask key questions.
 - **15-30** participants, depending on the organization's size.
2. Provide an **unlimited modeling surface** (at least the illusion):
 - A plotter **paper roll** on a **10-12** meter straight surface is usually enough
 - most whiteboards aren't wide enough
 - Bring plenty of sticky notes in all required colors and enough black markers for everyone.
3. Model **a whole business line** with domain events:
 - We do not focus on a specific area in Big Picture

Standard steps

Chaotic exploration

Every participant receives a stack of **orange sticky notes** and a **marker**. All at the same time, they write **domain events** and place them on the wall.

A **domain event** is anything meaningful that *happened* in the system, expressed as a verb in the past tense (e.g., Order Placed, Payment Received, Contract Signed).

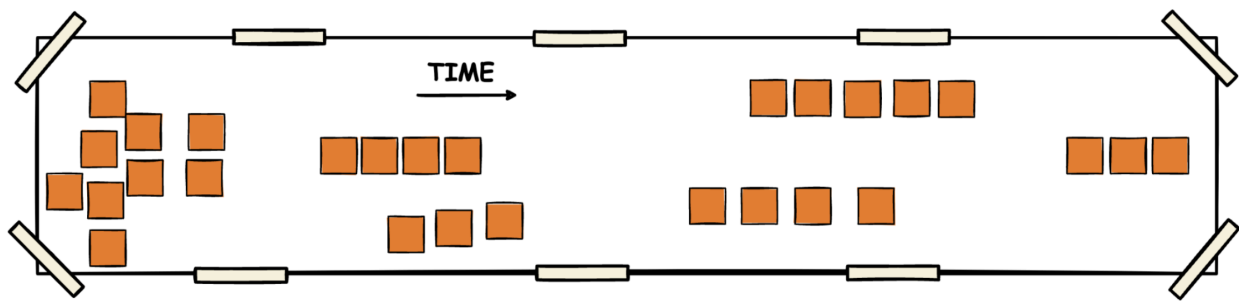
- The past tense anchors each event to a precise moment in time, often a state transition, making it possible to reason about sequences and causality later.
- Getting the phrasing right is hard at first, and that's fine. The goal is to start moving.

Each person works through their own mental model independently, placing notes that are **locally coherent** — ordered in their own head, connected to their own area of knowledge — with little relation to what others are putting up. The wall fills up in parallel, not as a shared conversation.

The outcome is deliberately **unstructured** and **chaotic**: different participants will contribute different levels of detail, granularity, and perspective. That mess is the point.

Some participants will dive in immediately. Others may hesitate, unsure of where to start. The facilitator's job at this stage is to keep people moving — by asking questions about their work — and to reassure participants that the chaos is intentional.

Resist the urge to clean up too early. Structure comes later.

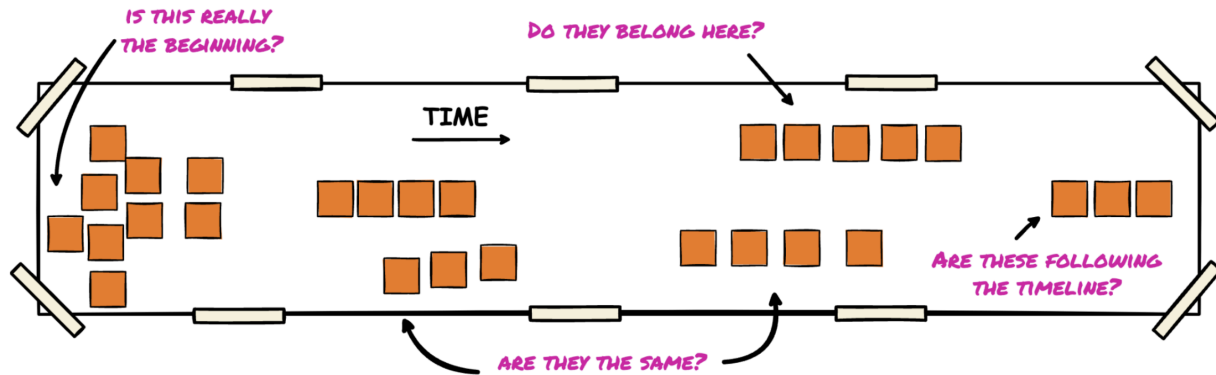


Enforce the timeline

The chaotic exploration left a mess on the wall. Now participants work together to arrange all domain events on a **single shared timeline**, from left to right.

This is where the first real conversations happen. Events that were written independently must now be reconciled — duplicates merged, contradictions surfaced, gaps filled.

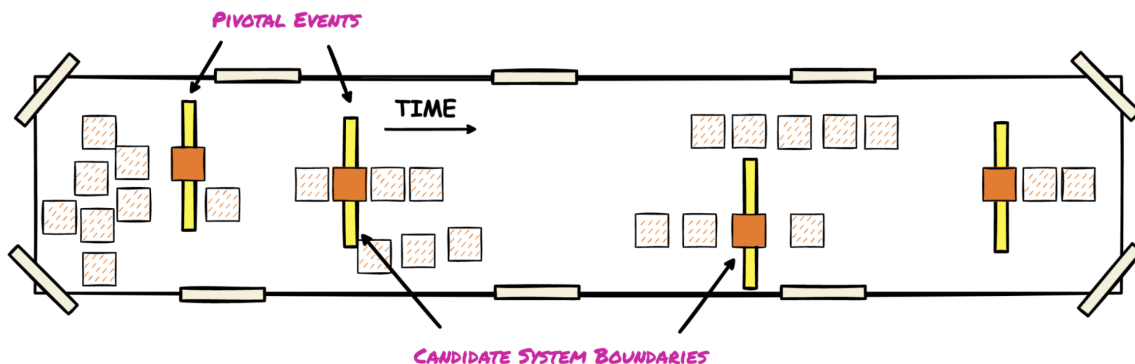
Participants will ask: “Which event comes first?” The answer: “Choose the most likely order. We’re not aiming for perfection.”



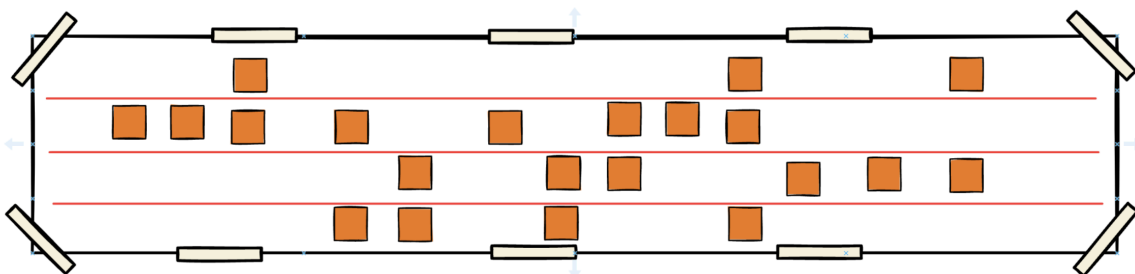
Pivotal events & Swimlanes

A flat sequence of events rarely holds for long. The facilitator introduces anchors to help the group organize the flow:

- **Pivotal Events** — key moments that mark a clear transition between phases (e.g., *Contract Signed*, *Payment Received*). Look for events that were written multiple times: duplicates signal importance. Mark them visually: a **strip of tape** works well in person. Everything else gets arranged relative to these anchors.



- **Swimlanes** — horizontal bands that separate parallel activities, roles, or departments. Use them where clarity matters more than wall space.



Most workshops combine both. Start with Pivotal Events, add Swimlanes where needed.

Hotspots

Not everything will be immediately clear. Some events will trigger disagreements, confusion, or questions that nobody can answer on the spot. These moments of friction are captured with **hotspots** — **magenta sticky notes**, placed deliberately *crooked* on the wall at the problem area.

A hotspot is not a failure. It is a signal: something here deserves a deeper conversation. It allows the group to acknowledge a problem without getting stuck on it, and move on.

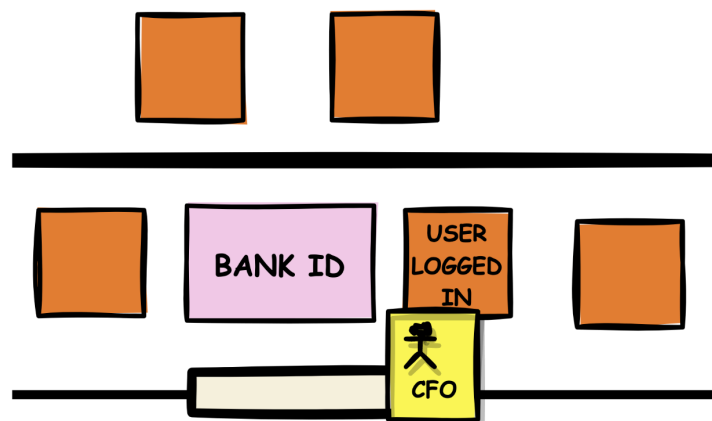
Hotspots emerge naturally from the conversation. The more honest the participants, the more hotspots appear — and that is a good sign.

People & Systems

The timeline is taking shape, but the story is still thin. The next step is to make visible who and what is involved in the flow.

- **People** are represented with **small yellow sticky notes**. They can be a name, a job title, a team, anything that represents a human presence in the flow. The definition is intentionally loose.
- **Systems** are represented with **large pink sticky notes**. They capture everything else: software components, external services, offices, third-party entities. Again, intentionally loose.

Place them where they play a meaningful role in the flow. This step often surfaces new hotspots, and that is expected.



Explicit walkthrough

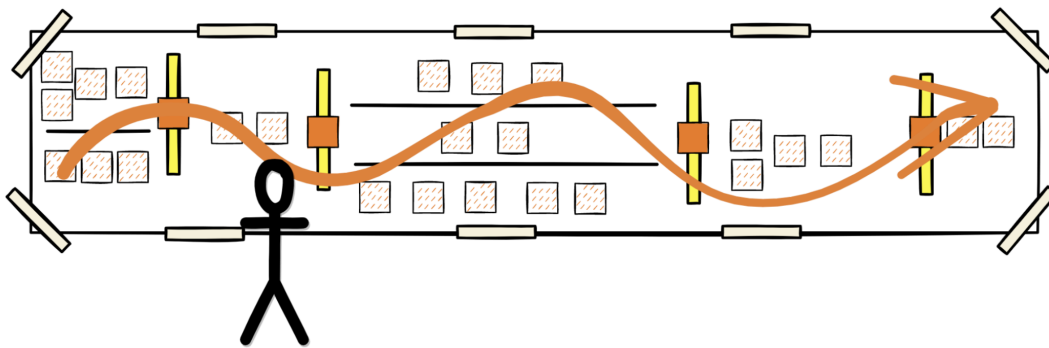
The board is populated. Now someone has to tell the story out loud.

A **narrator** walks along the timeline and tries to narrate the chain of events to the rest of the group. It doesn't have to go smoothly — and it probably won't. The narrator may not know the full story, may get things wrong, may hesitate. That is fine. The domain experts in the room will correct them.

This dynamic is one of the most interesting moments of an EventStorming session. Someone attempts to tell a story they don't fully know, and the audience helps — especially when something is clearly wrong. The **model is not born right**. It is born **visibly wrong**, and it *progressively gets closer to right* because everyone contributes to correcting it.

New hotspots appear. Events get rearranged. New Pivotal Events and Swimlanes emerge. The structure that started as chaotic begins, slowly, to make sense.

Narrators rotate every 5–10 minutes. When the walkthrough is complete, do a final pass in reverse — from end to start. Reading the timeline backwards often reveals missing events. Close with a simple question: “*Is anything still missing?*” Expect a few more additions.



Custom steps

You may pick the ones that are more suitable for your scenario.

- **Playing with value:** visualize and discuss the steps and activities that deliver or destroy value for your customers, employees, stakeholders, etc.
- **Problems and Opportunities:** explore explicit issues with the current model and propose solutions to address open problems or new ideas to tackle opportunities.
- **Arrow Voting:** quickly visualize through blue arrow stickers the convergence on a specific issue, which is probably your organization's current main blocker.
- **Extracting User Stories:** common step in Big Picture EventStorming during a software development project kick-off.
- **Extracting Bounded Contexts:** perfect homework for your technical team to derive a **Context Map** from your EventStorming exploration.
- **Visualizing ownership:** may make the connection between your flow and your development teams' areas of responsibility evident.

Outcome

A Big Picture session leaves the room with many outcomes:

- **A visible process.** Everything that could go wrong is on the wall. The most important constraints to the business are no longer invisible — they can be pointed at, discussed, and prioritized.

- **Massive learning across silos.** Developers learn about the domain. And the little secret is that the business actually learns about its own business too.
- **Consensus around the core problem.** At the end, participants vote with blue arrows on what they believe is the most critical blocker. The winner is selected by everyone in the room.
- **A feeling of clarity.** “*This is what we do, now I get it.*” Often, this is the most valuable thing that comes out of the session.
- **Early hints at system boundaries.** For software teams, the model already contains seeds of *bounded contexts*: the same people appearing in different parts of the wall, events that mark a clear handover between responsibilities. The business may not care about this yet — but the technical team now has a map.

Process Modeling EventStorming

The Process Modeling format is used for discovering the intricacies of *complex business processes* and for designing new ones. It is built around the idea of **collaborative games**: the team works together against a shared opponent, which is the complexity of the process itself. The facilitator should enforce this.

The scope is narrower: a single, end-to-end business process, with a clear starting point and a clear goal (usually the one voted at the end of the Big Picture format).

The group is smaller — typically **4–8 people** — and works together sequentially, rather than contributing in parallel.

Process Modeling enforces structure from the start. The goal is not just to understand *what* happens, but to model *how* it happens, step by step, decision by decision.

Assumptions

- We’re working on the most compelling problem
- The problem has a solution in software and/or process
- Key experts are available:
 - Business people, from the departments involved
 - Technical experts: software and service/UX designers
 - How to make them cooperate towards a common goal? Idea: **cooperative game**, with its own rules (explained later).

Assumptions can be broken if you are aware of the consequences.

The Team

Process Modeling (and Software Design) EventStorming require an **interdisciplinary team** with **complementary attitudes**. You want people from the business, from software design, from UX and service design — but the composition that matters most is not about roles. It is about *attitudes*.

Three archetypes make the game work:

- **The driller** constantly asks “what if?”, pushing the team into corner cases, alternative paths, and edge scenarios that the happy path tends to hide.
- **The pragmatic** knows which path delivers more value and why. They keep the team grounded when the discussion risks becoming too abstract.
- **The empathic** cares about emotions and feelings. They are the ones who notice that a rejection message, if poorly designed, can turn into a lost customer.

Notation

Process Modeling builds on the Big Picture notation with three additional elements:

- **Commands** represent an intent to change the system’s state. It is a **decision** taken by a user (or a piece of software). They are phrased as instructions, typically **verb + noun** (e.g., *Place Order*, *Cancel Subscription*, *Schedule Interview*).
- **Policies** are the glue between an **Event** and a **Command**. They capture the business rule that connects them: *whenever this happens, we do that*. A policy can be automatic or require human intervention. The language of a policy often follows the pattern: *whenever X, then always Y*.
- **Read Models** hold the information a person needs to make a decision before issuing a command. They make explicit what is often invisible: what data is available, where it comes from, and what assumptions are being made.

Grammar

Process Modeling introduces a **mandatory grammar**: a *repeating pattern* that structures the entire flow.

In Big Picture, structure emerges gradually from chaos. In Process Modeling, structure is **enforced from the start**.

The grammar can be read like this:

“an **Event** triggers a **Policy**. The policy leads to a **Command** which is executed by a **System**, and produces a new **Event**.”

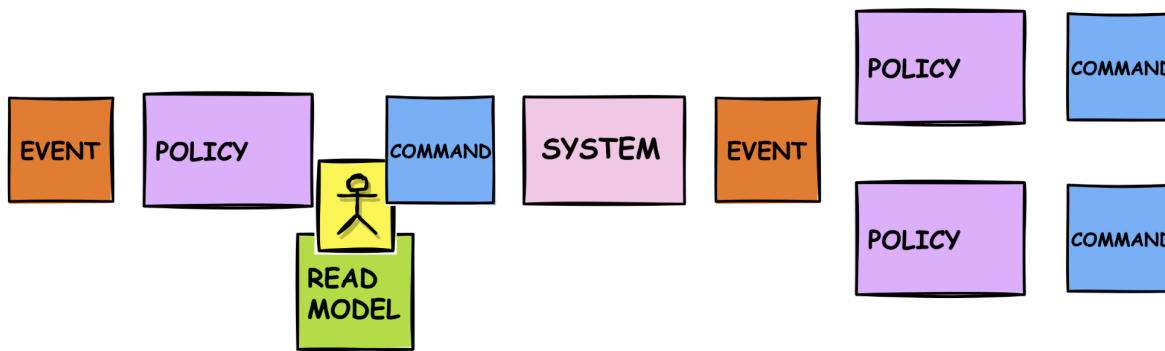
The pattern then repeats.

Event → **Policy** → **Command** → **System** → **Event**

When a human is involved, the flow expands slightly: before issuing a command, a person needs information to make a decision. That information is captured by a **Read Model**.

Policy → [**Human**] → [**Read Model**] → **Command**

The **grammar forces the team to ask tough questions**: *Does this always happen automatically? Who decides? What information do they need to decide?* Policies, in particular, are where the real behavior of an organization hides — and where the official narrative tends to diverge from reality.



Game Rules

1. Every **path** should be **completed**.
 - Incomplete paths hide assumptions and edge cases that will eventually cause problems.
 - Following every branch to a terminal state surfaces the decisions that nobody has made yet.
2. The **grammar** must be **respected**.
 - The repeating pattern is not just a notation convention — it is a thinking tool.
 - When the grammar breaks, it means the team has not yet understood that part of the process. The grammar makes gaps visible.
3. Every **stakeholder** should be **reasonably happy**.
 - If a stakeholder looks at the board and says "*this is not how it works*," the model is wrong — regardless of how internally consistent it may be.
 - Every step can create or destroy “value” (e.g., time, safety, reputation, money, happiness, engagement) for given users. Pick one you want to optimize for.
4. Every **Hotspot** should be **addressed**.
 - Dissent and disagreement can be made visible, but the team should try to find a possible solution for as many issues as possible.
 - A hotspot left completely unaddressed is a part of the process the team has chosen not to understand.

Outcome

- **A model of how the process actually works.** Not how it is supposed to work, but how it *does* work.
- **Explicit policies.** Decisions that were made informally — by habit, by convention, by whoever happened to be available — are now visible on the wall. They can be challenged, improved, or automated.
- **A foundation for implementation.** For software teams, the model maps almost directly to code. Commands become features. Policies become business logic. Read Models become queries. The conversation about design has already happened — on the wall, in plain language, with everyone in the room.

Software Design EventStorming

The Software Design EventStorming aims to *design software models* to support complex business processes.

The format expands the same collaborative game, with Domain-Driven Design in mind.

The event-driven nature of the exploration makes it an effective modeling tool for identifying **Aggregates** by zooming into individual Bounded Contexts discovered during the Big Picture EventStorming.

Game Rules

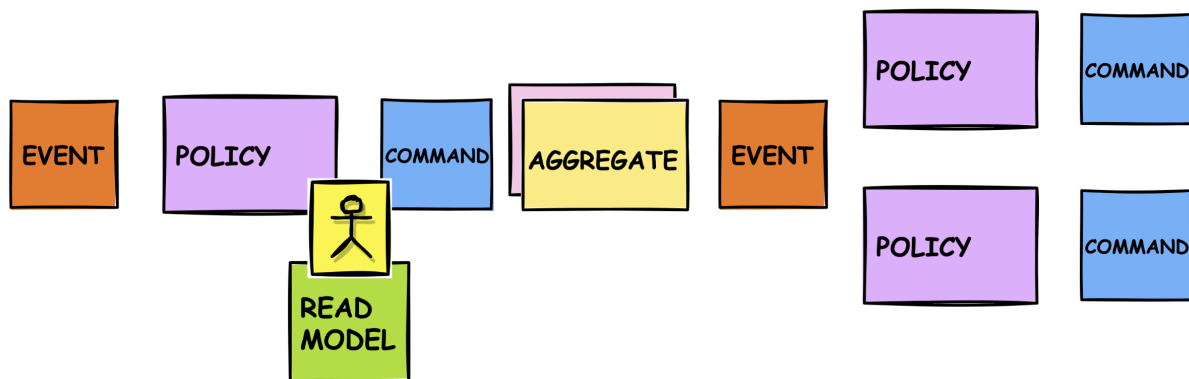
Software Design EventStorming is still a collaborative game. It inherits the four rules from Process Modeling and extends them with two more:

5. **Boundaries should be visualized.** Making boundaries explicit forces the team to have the conversation about where one model ends and another begins.
6. **Aggregates should be coherent.** An Aggregate is not a data container. It is a component with a lifecycle, and that lifecycle must be internally consistent from creation to termination. Any inconsistency is a signal that the model needs further refinement.

Defining Aggregates

In Process Modeling, every external service, platform, or component is represented by a **pink System sticky note**: a black box we interact with but do not own. In Software Design, the team starts making deliberate choices: which of these systems will we design and own ourselves, and which will remain external dependencies?

When a system moves from external to internal, its pink sticky note is replaced by a **yellow Aggregate**. This small visual change carries a significant meaning: we are no longer treating this as a black box. We are taking responsibility for it.



Both pink **Systems** and yellow **Aggregates** take **Commands** and emit **Events**. The difference is ownership, and ownership means we can change it.

Identifying Aggregates can be challenging. The typical mistake is to start by *naming* them (e.g., *Order*, *Customer*, *Invoice*) and then fill them with commands and events. This produces data containers, not behavioral components.

The right approach is the opposite: let the behavior define the Aggregate. Look at the commands and events already on the wall, find clusters of coherent behavior, and let the name emerge from that.

Once the Aggregates start taking shape, the team works to make their boundaries explicit:

1. **Isolate and consolidate.** Move all yellow Aggregate sticky notes to a clean area. If the same concept appears multiple times, merge them into a single, canonical Aggregate.
2. **Define the boundaries.** For each Aggregate, collect all the Commands it handles on one side, and all the Events it emits on the other. This defines its public interface: what it accepts and what it produces.
3. **Check for completeness.** Review each Aggregate for missing lifecycle steps. If there is an *Update Shipping Address* command but no *Create Order*, an important starting point has been missed.

Bounded Contexts

A **Bounded Context** is a self-contained part of the system with its own language, its own model, and its own team.

The boundaries often reveal themselves through linguistic mismatches: different departments using different words for the same concept, or the same word for different concepts. These are signals that mark where one Bounded Context ends and another begins.

Forcing a single unified model across the entire system creates friction. Each Bounded Context works better as an independent model, optimized for the language and purpose of the team that owns it.

Software Design EventStorming is also a great place to distill the **Ubiquitous Language** of each Bounded Context, and to stress-test the model against complex real-world scenarios that can later be turned into acceptance tests.

Outcome

- A left-to-right flow where boundaries and key building blocks start to emerge. **Aggregates** are shaped by their *behavior* and *role* in the end-to-end interaction, looking more like state machines than data containers.
- Bounded Contexts are now precisely highlighted. Their purpose reflects into the lifecycle of the moving parts, offering options for smarter decomposition.
- Another significant outcome is likely in the trash can: ideas we tried and discarded, wordings that turned out naive after closer scrutiny, and Hot Spots we resolved in a previous iteration.

Remote EventStorming

Brandolini has consistently maintained a **skeptical stance** toward remote formats, though the COVID-19 pandemic forced him to reconsider some of his positions.

Big Picture was deemed *essentially incompatible* with remote delivery.

Skepticism is rooted in the fact that this format depends on a very specific set of conditions that simply cannot be replicated online:

1. **Parallel Conversations:** remote bandwidth supports less than one meaningful conversation at a time, making the simultaneous multi-threaded discussions that drive Big Picture discovery impossible.
2. **Localized Conversations:** people naturally cluster around the relevant part of the model in person, building awareness and relationships. This organic dynamic does not occur remotely.
3. **Body Language:** without facial expressions and posture to read, facilitators and participants lose a critical source of information, making disagreement and disengagement far harder to detect.
4. **Active Facilitation:** the facilitator acts like a DJ reading the room. Remotely, most micro-impediments go invisible and unaddressed.
5. **Full Immersion:** the peer pressure and shared commitment of a physical room vanish remotely, making partial engagement and distraction far more likely.
6. **Full Day Format:** sustaining a full-day conference call is simply not realistic, limiting the depth of exploration Big Picture is designed to achieve.
7. **Interruption-Free Environment:** being online exposes everyone to notifications and distractions that a paper-based physical workshop naturally blocks out.
8. **No Scope Limitation:** when everything moves more slowly remotely, the time-box guarantee breaks down and relevant parts of the flow risk being skipped.
9. **Moving Around:** walking the modeling surface keeps energy up and contributes to immersion. This is entirely absent in a remote setup.
10. **The Punch:** Big Picture builds toward a revelatory moment designed to land with maximum impact. Remotely, this effect is lost entirely.

The story is different for **Process Modeling** and **Software Design** formats.

The two workshop rely on a **smaller number of participants**, hopefully from an **interdisciplinary background** (e.g., software, business, UX specialists), shorter sessions, and a collaborative game dynamic that can partially survive the transition to digital tools.

However, significant value is still lost:

- **Process Modeling.** Brandolini's own experimental remote session revealed three core problems:
 1. Body language was deeply missed, leaving participants without the basic confirmation of a nod or a smile.
 2. Structure emerged slowly: actions like spotting duplicates or grouping concepts require explicit verbal negotiation instead of quick, silent physical gestures.
 3. Participants talked less, even when voice was their only remaining communication channel.
- **Software Design.** This format tolerates the transition slightly better, for three reasons:
 1. Participants are typically software professionals, already accustomed to online collaboration.
 2. The scope is narrower and sessions can be kept to around 90 minutes.
 3. The focus is more technical, reducing the need for broad interdisciplinary dynamics.

EventStorming in COVID-19 times

When the pandemic made in-person workshops impossible, Brandolini revised his position and outlined several practical mitigations to make remote sessions more viable:

- **Clarify the purpose upfront:** without the flexibility of a physical room, the workshop goal must be defined clearly in advance and the steps designed around it.
- **Seed the structure:** instead of relying on fully emergent organization, provide some initial structure (Pivotal Events or Temporal Milestones) to reduce the impact of the initial stand-off.
- **Color-coding per person:** compensates for the loss of handwriting as a personal signature, making individual contributions identifiable and lazy participants visible.
- **Mob modeling rounds:** time-boxed leadership slots of 5–7 minutes, where one person leads the modeling uninterrupted, keeping the session focused and structured.
- **Make disagreements explicit:** without facial expressions and body language, disagreement must be surfaced through visible tools like comments or digital hotspots.
- **Inspect and adapt checkpoints:** regular reflection moments to assess what is working and correct course, replacing the implicit feedback that body language would normally provide.
- **Multiple communication channels:** using chat alongside voice helps keep conversation parallel and traceable, partially compensating for the loss of bandwidth.

Despite these adaptations, his core conclusion remained unchanged: **remote sessions are a necessary compromise, not a true replacement**, and facilitators should always be transparent about what is inevitably lost in the process.

References

[EventStorming: The most versatile Collaborative Modelling format](#)

[Event Storming - Alberto Brandolini - DDD Europe 2019](#)

[Event Storming: 50.000 Orange Stickies Later • Alberto Brandolini • GOTO 2018](#)

[Event Storming with Alberto Brandolini \(Alberto Brandolini\)](#)

[Event Storming – The Complete Guide](#)

[Remote EventStorming](#)

[EventStorming in COVID-19 times](#)